



## Summer of Bitcoin 2026

# Rust SDK for ContextVM
# Project Proposal By

## Harsh Chandwani

| | |
|---|---|
| **Email Address** | harsh9425181074@gmail.com |
| **Github** | https://github.com/harsh04044 |
| **Linkedin** | https://www.linkedin.com/in/harsh-chandwani-b9a48238a/ |
| **Phone** | **+91 9425181074** |
| **TimeZone** | **Indian Standard Time(IST) (GMT + 5:30)** |
| **Discord** | **harsh040444 (username)** |
| **University** | **Indian Institute of Technology Madras** |
| **Idea Link** | IDEA LINK FOR THE PROPOSAL **(Rust SDK for ContextVM)** |

# INDEX


1. Overview and Rationale 3

2. Current State of the rs-sdk 4

3. Goals and Expected Outcomes 5

4. Detailed Architecture 10

5. Technical Implementation Approach 15

6. Implementation Steps and Timeline 18

7. Security and Privacy 21

8. Testing Strategy 22

9. Documentation Plan 23

10. Competency Test: Prior Contributions 24

11. Benefits to the Community 25

12. About Me 26

13. References 26

14. Availability & Commitment 27

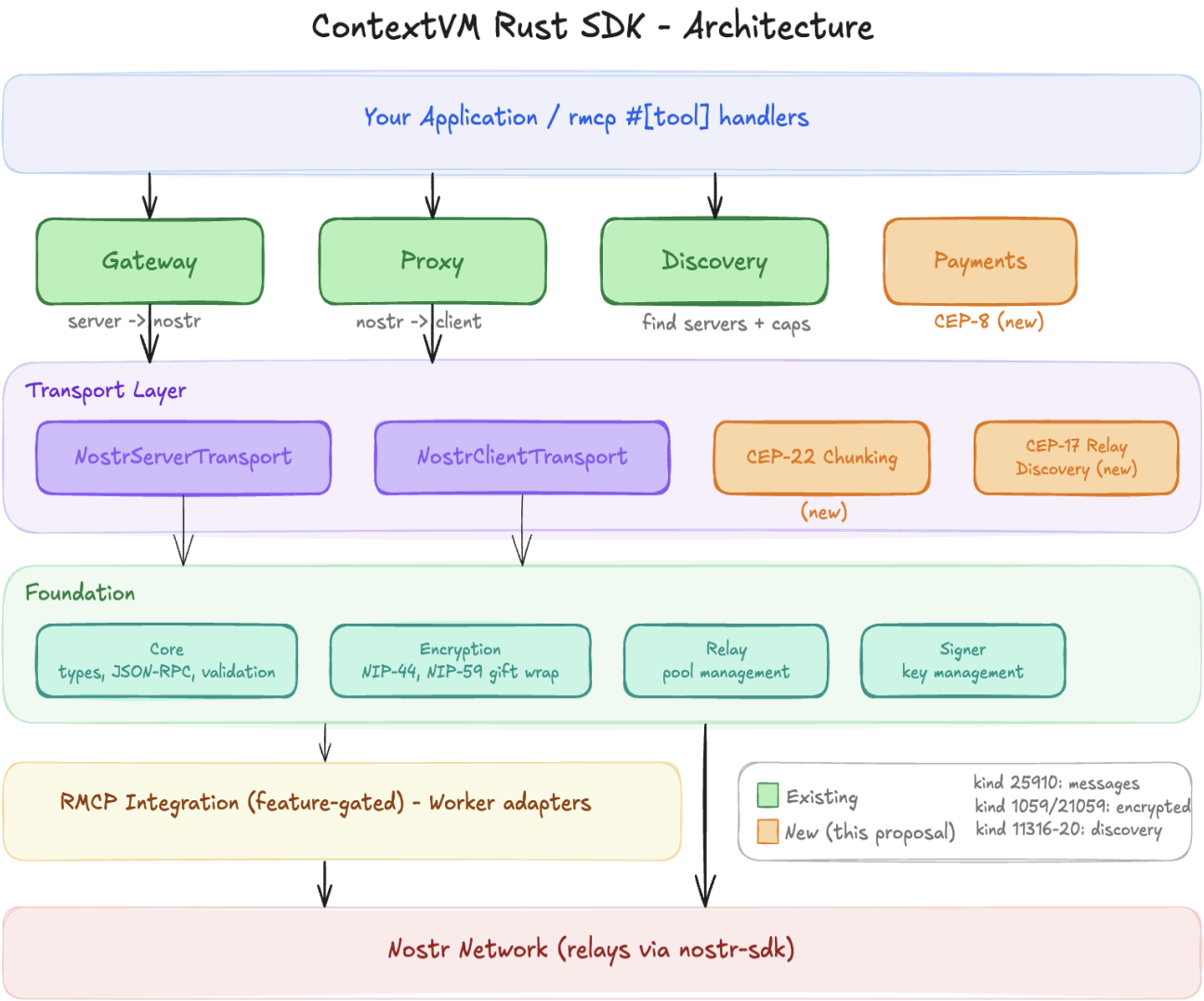
15. Post SoB 27

# 1. Overview and Rationale

ContextVM is a protocol that bridges MCP (Model Context Protocol) with the Nostr network, that lets people access computational services in a decentralized way. It allows MCP servers (which expose tools, resources, and prompts) to communicate through Nostr's distributed relay infrastructure without requiring domains, inbound ports, or OAuth. Messages are JSON-RPC 2.0 payloads inside Nostr events, with secp256k1 identity and optional NIP-44/NIP-59 encryption.

The rs-sdk is the Rust port of the ContextVM TypeScript SDK (the reference implementation). It aims to achieve full feature parity, making Rust a first-class implementation target for the ContextVM protocol. The project reuses MCP's official Rust SDK (rmcp crate) for protocol semantics and nostr-sdk for Nostr operations.

Building a complete Rust SDK for ContextVM delivers the following advantages:



- **High-Performance MCP Gateways:**
  - Gateway servers managing hundreds of concurrent MCP sessions need predictable latency. GC pauses in Node.js can stall all active sessions at once, which is a real problem for always-on gateways. Rust with tokio gives you concurrent session handling without that risk.
- **rmcp Ecosystem Integration:**
  - The rs-sdk uniquely integrates with the official MCP Rust SDK (rmcp crate) via Worker trait adapters. Any Rust MCP server using rmcp's #[tool] macros can be exposed over Nostr with zero additional protocol code. This is a capability the TS SDK cannot offer.
- **FFI Foundation:**
  - A well-structured Rust crate can be wrapped via UniFFI (mobile), PyO3 (Python), and napi-rs (Node.js), bringing ContextVM to ecosystems beyond TypeScript and Rust.

## 2. Current State of the rs-sdk

The rs-sdk is approximately 70% feature-complete relative to the TypeScript SDK. The core protocol, transport layer, gateway, proxy and discovery modules are fully implemented with 120 tests across 26 source files. I have been an **active contributor** over the past month, with **12 merged PRs**.

### Implemented Features

| Feature | Notes | Status |
|---|---|---|
| Core (constants, types, errors, serializers, validation) | Full JSON-RPC 2.0 type system, 1MB validation | **Complete** |
| NIP-44 Encryption + NIP-59 Gift Wrap | Single-layer gift wrap, TS SDK interop verified | **Complete** |
| Client Transport | Request correlation, stateless mode, encryption negotiation | **Complete** |
| Server Transport | Multi-client sessions, cleanup, auth, progress tokens | **Complete** |
| Gateway + Proxy | Bidirectional MCP ↔ Nostr bridging | **Complete** |
| Discovery (kinds 11316–11320) | Servers, tools, resources, prompts, deletion | **Complete** |
| RMCP Integration (feature-gated) | Worker adapters for rmcp Service<Role> | **Complete** |
| LRU Gift-Wrap Deduplication | 5000-entry LRU cache | **Complete** |
| CEP-6 Public Announcements | Kind 11316 publication | **Complete** |

### Feature Parity Gap

The following features are implemented in the TypeScript SDK but missing or incomplete in the rs-sdk:

| Gap | rs-sdk Status |
|---|---|
| CEP-8 Payments (NWC, LNbits, Zaps) | **Missing** |
| CEP-22 Oversized Payload Transfer | **Missing** |
| CEP-23 Server Profile Metadata | **Missing** |
| CEP-17 Relay Discovery (client logic) | **Partial** |
| CEP-19 Ephemeral Gift Wrap (send) | **Partial** |
| Per-Client Gateway (LRU isolation) | **Missing** |
| Middleware Chain | **Missing** |
| Request Timeout Enforcement | **In-flight PR** |
| Unauthorized Error Responses | **In-flight PR** |

## 3. Goals and Expected Outcomes

### A. Closing the Feature Parity Gap

- **CEP-8 Payment Module**
  - **Objective:** Build the complete payment module supporting Lightning Network micropayments for priced MCP capabilities, matching the TS SDK's payments/ module.
  - **Key Functions:**
    - **Defining PaymentHandler and PaymentProcessor traits** with async methods for invoice creation, payment, and verification.
    - **Implementing withClientPayments () and withServerPayments ()** transport wrappers that intercept payment_required notifications and handle the payment flow automatically.
    - **Building NWC (NIP-47) handler/processor** using the nwc crate from rust-nostr for Nostr Wallet Connect integration.
    - **Building LNbits REST handler/processor** for HTTP-based Lightning invoice creation and payment via LNbits API.
    - **Implementing NIP-57 zap receipt processor** for server-side zap-based payment verification.
    - **Constructing cap tag and pmi tag builders** for advertising priced capabilities on server announcements.

- **CEP-22 Oversized Payload Transfer**
    - **Objective:** Implement chunked payload transfer for messages exceeding 48KB, enabling large tool responses and resource transfers over Nostr's relay infrastructure.
    - **Key Functions:**
        - **Building a sender module** that splits oversized payloads into chunks with sequence numbers and publishes them as notifications/progress frames.
        - **Building a receiver module** that reassembles chunks in order, validates total size, and delivers the complete message to the application layer.
        - **Implementing session management** that tracks active transfers with unique transfer IDs, handles timeouts and supports abort frames for cancellation.
        - **Integrating into transport layer** so messages exceeding 48KB automatically route through the chunking pipeline transparently.
- **CEP-17 Relay Discovery**
    - **Objective:** Implement client-side relay discovery so clients can dynamically find the correct relays for a target server, rather than relying solely on hardcoded relay URLs.
    - **Key Functions:**
        - **Implementing fetchServerRelayList ()** that queries bootstrap relays for a server's kind:10002 event, subscribes to { kinds: [10002], authors: [serverPubkey] }, waits for EOSE, and extracts r tags.
        - **Implementing selectOperationalRelayUrls ()** that picks the minimal relay set from the discovered list, preferring unmarked relays and falling back to union of read+write marked relays.
- **CEP-19 Ephemeral Gift Wrap (Send-Side)**
    - **Objective:** Complete ephemeral gift wrap support by adding kind:21059 event creation. Currently the rs-sdk can only receive ephemeral gift wraps; this adds send-side support.
    - **Key Functions:**
        - **Implementing ephemeral gift wrap creation** in the encryption module, constructing kind:21059 events with NIP-44 encrypted content and ephemeral sender keys.
        - **Adding GiftWrapMode configuration** to transport options, allowing users to choose between persistent (kind 1059) and ephemeral (kind 21059) wrapping.

- **CEP-23 Server Profile Metadata**
  - **Objective:** Implement server profile metadata publication via kind:0 events with NIP-01 profile fields, allowing servers to publish human-readable identity information.
  - **Key Functions:**
    - **Implementing publishProfileMetadata ()** in the server transport that publishes a kind:0 event with name, about, picture, and website fields.
    - **Supporting both public and private servers** profile metadata is published regardless of announcement visibility.

## B. Extending the Authorization Model

- **isPubkeyAllowed Callback Extension**
  - **Objective:** Extend the authorization callback to support fine-grained, per-method authorization by passing additional context about the requested capability. This extension has been validated by gzzz for both SDKs.
  - **Key Functions:**
    - **Defining an AuthorizationContext struct** containing client_pubkey, method (e.g., “tools/call”), and capability_name (e.g., the tool name).
    - **Updating the callback signature** from Fn(&str) -> bool to Fn(AuthorizationContext) -> bool, passing the full context.
    - **Wiring the context through the server transport’s authorization flow,** extracting method and capability name from incoming JSON-RPC messages before calling the callback.

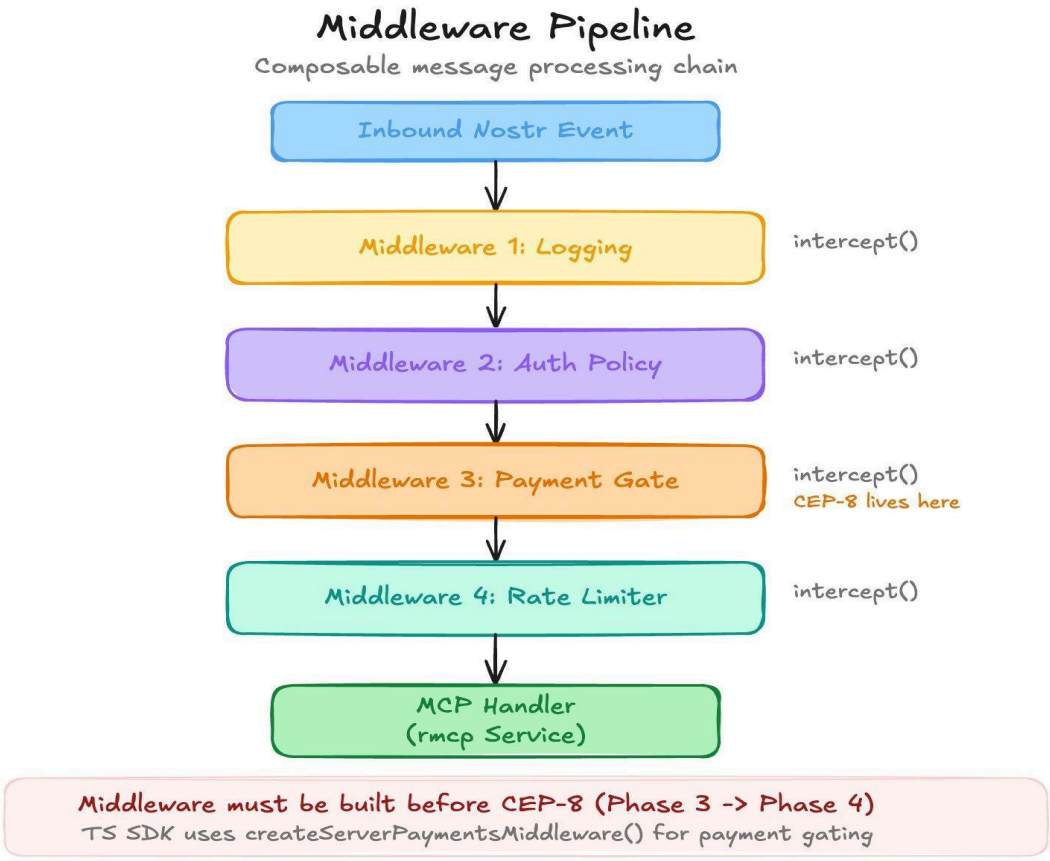
## C. Test Coverage

- **Test Infrastructure**
  - **Objective:** Build practical test infrastructure and achieve solid coverage for all new features, to make sure both implementations stay compatible.
  - **Key Functions:**
    - **Landing MockRelayPool (PR #30):** A trait-based relay abstraction so tests can run without live WebSocket connections. Foundation for all integration testing.
    - **Porting conformance tests:** From the TS SDK’s 37-file test suite, covering wire format compatibility, encryption roundtrips, and protocol flow correctness.
    - **Building per-feature test suites:** Each new CEP implementation includes unit tests (module-level), integration tests (cross-module via MockRelayPool), and conformance tests (wire format compatibility).

        - **Creating FakePaymentHandler/FakePaymentProcessor:** Test stubs for CEP-8 payment flows, enabling payment testing without Lightning infrastructure.

## D. Per-Client Gateway and Middleware

- **Per-Client Gateway**
    - **Objective:** Implement per-client MCP state isolation in the gateway, mirroring the TypeScript SDK's per-client mode.
    - **Key Functions:**
        - **Maintaining isolated MCP session state per Nostr pubkey,** managed via an LRU cache with configurable maximum size and eviction callback.
        - **Spawning per-client transport instances** so that each client's MCP initialize/tool-call state is fully isolated.
- **Middleware Chain**
    - **Objective:** Implement a composable message processing pipeline for pluggable transport extensions.
    - **Key Functions:**
        - **Defining a Middleware trait** with intercept methods for inbound and outbound messages.
        - **Implementing stack composition** so middleware layers (logging, rate limiting, payment gating) can be chained without modifying core transport code.




## E. FFI Readiness and Language Bindings Note

- **Crate Restructuring and FFI Strategy**
  - **Objective:** Restructure the crate for clean FFI boundaries and deliver a document evaluating cross-language binding strategies.
  - **Key Functions:**
    - **Separating into contextvm-core (transport-agnostic)** and contextvm-nostr (Nostr-specific transport). This enables FFI wrappers to depend only on the core crate without pulling in nostr-sdk.
    - **Evaluating binding tools:** UniFFI (Kotlin/Swift for mobile, used in Firefox), PyO3 (Python with async support via pyo3-async-runtimes), napi-rs (Node.js with WASM support), cbindgen (C/C++ headers).
    - **Documenting API surface recommendations:** Which types to expose across FFI boundaries, how to handle async, opaque handle patterns for stateful objects.

# 4. Detailed Architecture

## High-Level Design

The rs-sdk follows a layered architecture mirroring the TypeScript SDK while using Rust-specific patterns (traits, tokio async, feature gates):

## RMCP Integration

The rs-sdk uniquely integrates with the official MCP Rust SDK (rmcp v0.16.0) via an adapter pattern:

- **NostrServerWorker / NostrClientWorker**
  - Implement rmcp's Worker trait, bridging Nostr events to rmcp's typed service model.
  - Any Rust MCP server using rmcp's `#[tool]` macros works over Nostr with zero changes.
- **Conversion boundary (convert.rs)**
  - Uses serde JSON round-tripping between the SDK's JsonRpcMessage types and rmcp's role-parameterized types.
  - Since both sides are JSON-RPC 2.0, the wire shape is identical.
- **Design decisions**
  - **Single-client binding:** NostrServerWorker binds to the first client pubkey (rmcp services are single-peer). Multiple clients require multiple worker instances.
  - **Correlation map:** Maps rmcp's response IDs back to Nostr event IDs for proper response routing.
  - **Lifecycle ownership:** `Worker::run()` owns the transport lifecycle via `tokio::select!` over cancellation, inbound Nostr, and outbound rmcp channels.

### Middleware Trait (proposed)

Messages pass through a composable middleware chain before reaching the MCP handler. This is the architectural foundation that CEP-8 payments will build on:

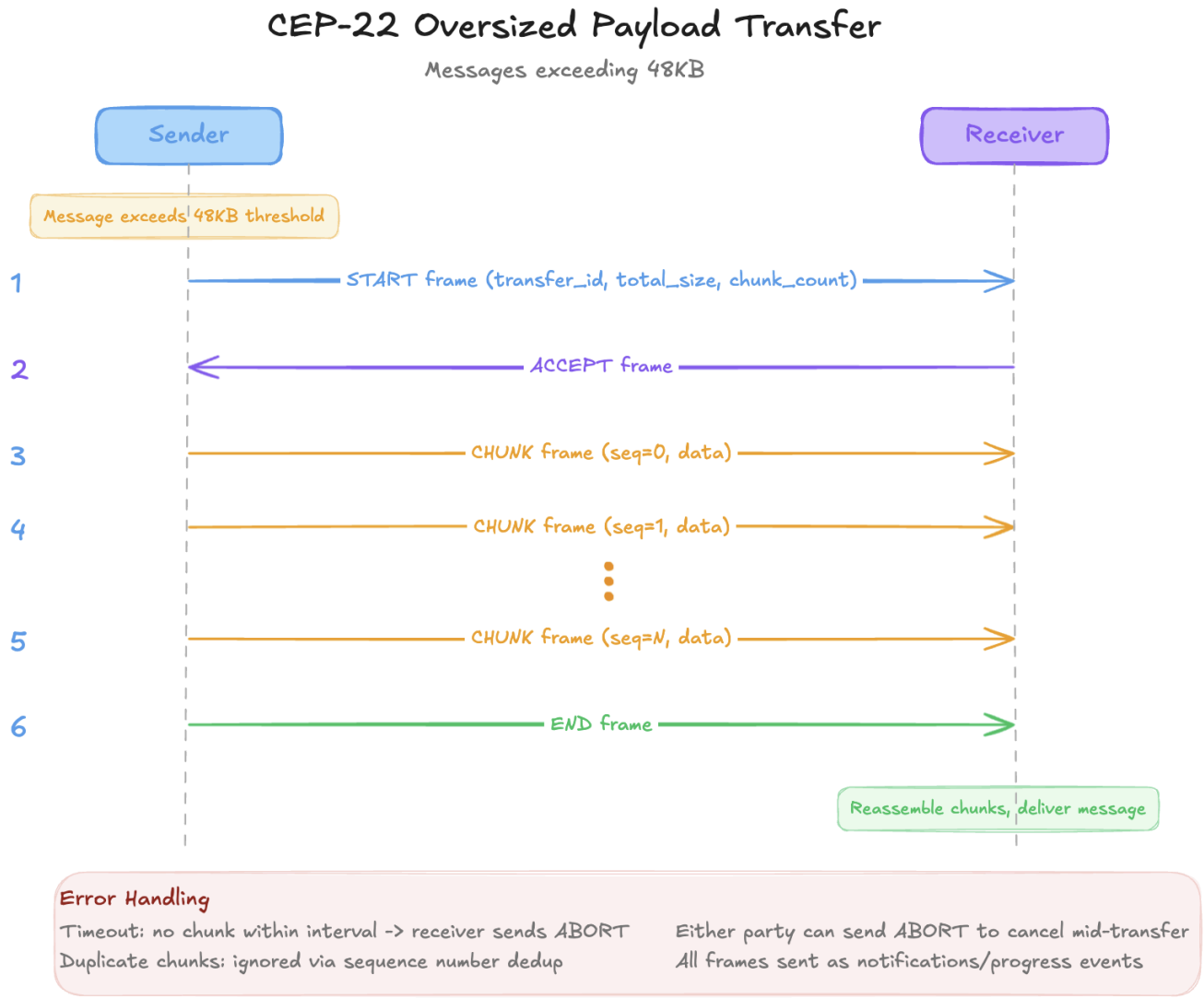
```rust
/// Context passed to each middleware layer.
pub struct MiddlewareContext {
    pub client_pubkey: String,
    pub client_pmis: Option<Vec<String>>,
}

/// Inbound middleware for composable message processing.
///
/// Mirrors the TS SDK's `InboundMiddlewareFn`. Each middleware
/// calls `forward` to pass to the next layer, or drops the message.
#[async_trait]
pub trait InboundMiddleware: Send + Sync {
    async fn intercept(
        &self,
        message: JsonRpcMessage,
        ctx: &MiddlewareContext,
        forward: &dyn Fn(JsonRpcMessage) -> BoxFuture<'_, Result<()>>
            + Send + Sync,
    ) -> Result<()>;
}
```

## Data Flow: CEP-22 Oversized Transfer

- **Sender Flow**
  - Outbound message exceeds 48KB threshold.
  - Sender creates a transfer session with unique transfer_id.
  - Sender publishes a start frame (notifications/progress) with total size and chunk count.
  - Receiver acknowledges with an accept frame.
  - Sender splits payload into chunks, publishes each as a chunk frame with sequence number.
  - After all chunks sent, sender publishes an end frame.
  - Receiver reassembles chunks in order, validates total size, delivers complete message.
- **Error Handling**
  - **Timeout:** If no chunk arrives within configured interval, receiver aborts the transfer.

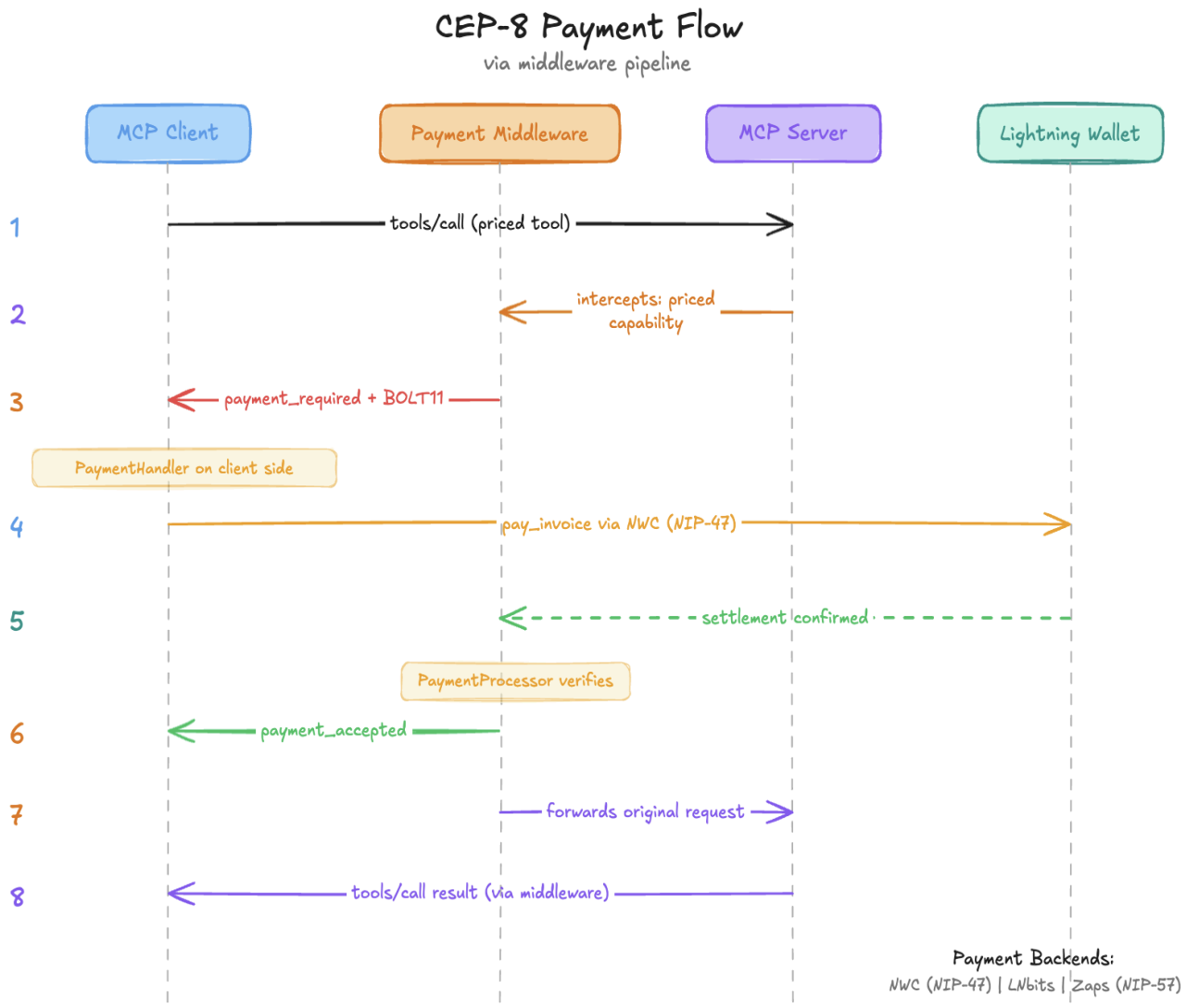
- **Missing chunks:** Receiver tracks sequence gaps and aborts if gaps cannot be filled.
- **Duplicate chunks:** Ignored based on sequence number deduplication.
- **Abort:** Either party can publish an abort frame to cancel mid-transfer.




## Data Flow: CEP-8 Payment Flow

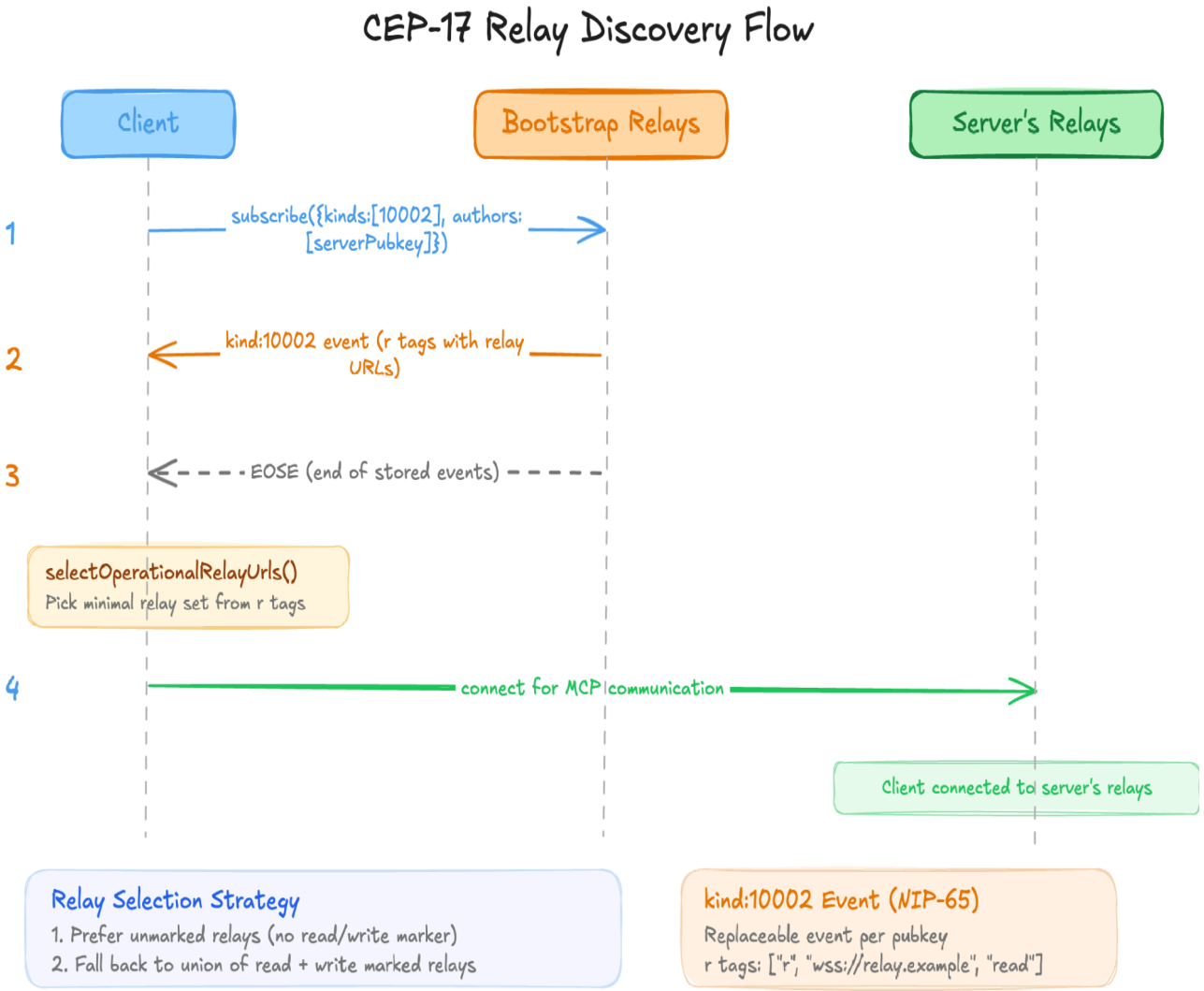
- **Server Advertises Pricing**
  - Server constructs cap tags listing priced capabilities (e.g., tools/call for tool_name costs 100 sats).
  - Server constructs pmi tags listing accepted payment methods (e.g., bitcoin-lightning-bolt11).

    - Tags are included in the server announcement event (kind 11316).
- **Client Requests a Priced Capability**
    - Client sends a normal JSON-RPC request (e.g., tools/call).
    - Server identifies the request requires payment.
    - Server sends notifications/payment_required with a BOLT11 invoice (pay_req field).
    - Client's PaymentHandler pays the invoice via NWC (NIP-47) or LNbits.
    - Server's PaymentProcessor verifies settlement (NWC confirmation or zap receipt).
    - Server sends notifications/payment_accepted.
    - Server processes the original request and returns the JSON-RPC response.



## Data Flow: CEP-17 Relay Discovery

- **Discovery Flow**
  - Client wants to connect to a server identified only by pubkey.
  - Client calls fetchServerRelayList (serverPubkey, bootstrapRelays).
  - Function connects to bootstrap relays and subscribes to { kinds: [10002], authors: [serverPubkey] }.
  - Waits for EOSE (end of stored events), picks the latest event by `created_at`.
  - Extracts r tags as relay entries (URL + optional read/write marker).
  - `selectOperationalRelayUrls()` picks the minimal relay set: prefers unmarked relays, falls back to union of read+write.
  - Client connects to the discovered relays for MCP communication.



# 5. Technical Implementation Approach

## Module Integration and Dependency Strategy

- **Core dependencies**
  - **nostr-sdk 0.43 (with nip59 feature):** Provides relay pool, NIP-44 encryption, NIP-59 gift wrapping, event building/signing, and key management. The **sdk** delegates all Nostr-level operations rather than reimplementing them.
  - **rmcp 0.16.0 (feature-gated):** Gives you typed message models, handler traits, and macros like `#[tool]` for declaring tools without boilerplate. It's behind a feature flag, so you can skip it if you just need the raw Nostr transport.
- **New dependencies for proposed features**
  - **nwc crate:** From rust-nostr ecosystem, for NIP-47 Nostr Wallet Connect in CEP-8 payments.
  - **reqwest (optional):** For LNbits REST API integration in CEP-8 payments. Feature-gated to avoid pulling HTTP stack when not needed.

## CEP-8 Payment Module Design

The payment module uses two traits: PaymentHandler (client-side) and PaymentProcessor (server-side). Three interchangeable backends implement these traits:

```rust
/// Payment handler trait (client-side: pays invoices)
#[async_trait]
pub trait PaymentHandler: Send + Sync {
    async fn pay_invoice(&self, bolt11: &str) ->
Result<PaymentResult>;
    fn supported_pmi(&self) -> &str;
}

/// Payment processor trait (server-side: creates & verifies)
#[async_trait]
pub trait PaymentProcessor: Send + Sync {
    async fn create_invoice(
        &self,
        amount_msats: u64,
        desc: &str,
    ) -> Result<String>;
    async fn verify_payment(&self, bolt11: &str) ->
Result<bool>;
    fn pmi(&self) -> &str;
}
```

| Backend | Client Handler | Server Processor |
|---|---|---|
| NWC (NIP-47) | LnBolt11NwcPaymentHandler | LnBolt11NwcPaymentProcessor |

| LNbits REST | LnBolt11LnbitsPaymentHandler | LnBolt11LnbitsPaymentProcessor |
|---|---|---|
| NIP-57 Zaps | - | LnBolt11ZapPaymentProcessor |

***Note:*** *LNbits backend is lower priority and will only be tackled if time permits after the core NWC and Zap backends are complete. First priority is to make rust sdk core ready.*

## CEP-22 Module Structure

CEP-22 frames are modeled as a tagged enum with serde, so they serialize to the same JSON shape as the TS SDK's OversizedTransferFrame union type:

```rust
/// CEP-22 oversized transfer frame types.
///
/// Carried as `notifications/progress` with the `cvm` field
/// containing the frame payload. Matches the TS SDK's
/// `OversizedTransferFrame` union type.
#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(tag = "frameType", rename_all = "lowercase")]
pub enum TransferFrame {
    Start {
        digest: String,
        total_bytes: u64,
        total_chunks: u32,
    },
    Accept,
    Chunk { data: String },
    End,
    Abort {
        #[serde(skip_serializing_if = "Option::is_none")]
        reason: Option<String>,
    },
}

/// Transfer policy limits.
pub struct TransferPolicy {
    pub max_transfer_bytes: u64,       // default: 100 MiB
    pub max_transfer_chunks: u32,      // default: 10,000
    pub max_concurrent_transfers: usize, // default: 64
    pub chunk_timeout: Duration,       // default: 30s
}
```

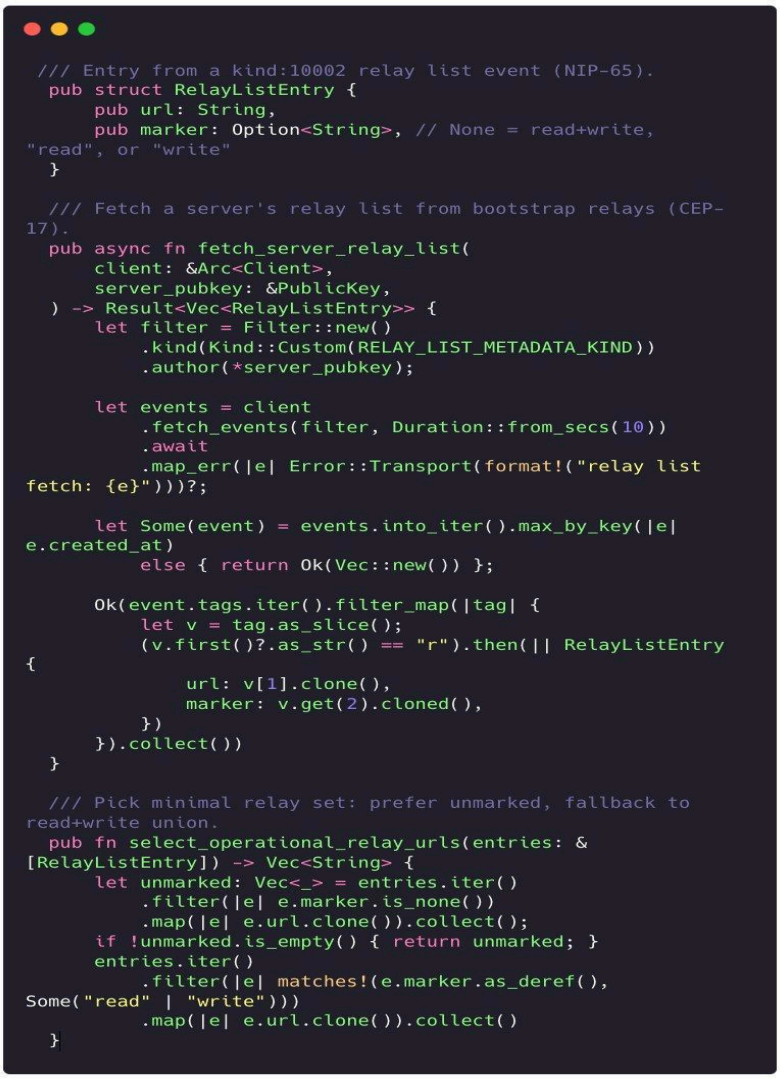
## isPubkeyAllowed Extension

The authorization callback gains request context so servers can make per-method decisions (e.g., allow tools/list but deny tools/call for expensive tools):

```rust
/// Proposed AuthorizationContext (extensible)
pub struct AuthorizationContext<'a> {
    pub client_pubkey: &'a str,
    pub method: Option<&'a str>,          // e.g. "tools/call"
    pub capability_name: Option<&'a str>, // e.g. tool name
}

/// Updated config field
pub is_pubkey_allowed: Option
    Box<dyn Fn(AuthorizationContext) -> bool + Send + Sync>
>,
```

## CEP-17 Relay Discovery Implementation



```rust
/// Entry from a kind:10002 relay list event (NIP-65).
pub struct RelayListEntry {
    pub url: String,
    pub marker: Option<String>, // None = read+write, "read", or "write"
}

/// Fetch a server's relay list from bootstrap relays (CEP-17).
pub async fn fetch_server_relay_list(
    client: &Arc<Client>,
    server_pubkey: &PublicKey,
) -> Result<Vec<RelayListEntry>> {
    let filter = Filter::new()
        .kind(Kind::Custom(RELAY_LIST_METADATA_KIND))
        .author(*server_pubkey);

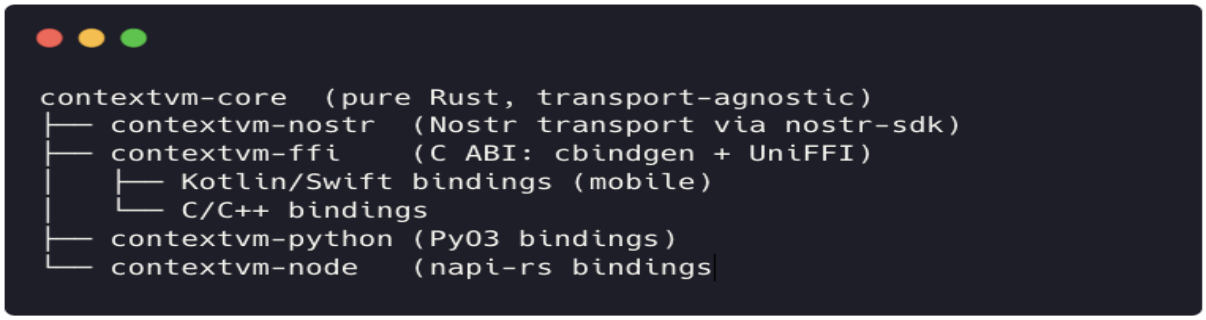
    let events = client
        .fetch_events(filter, Duration::from_secs(10))
        .await
        .map_err(|e| Error::Transport(format!("relay list fetch: {e}")))?;

    let Some(event) = events.into_iter().max_by_key(|e| e.created_at)
        else { return Ok(Vec::new()) };

    Ok(event.tags.iter().filter_map(|tag| {
        let v = tag.as_slice();
        (v.first()?.as_str() == "r").then(|| RelayListEntry {
            url: v[1].clone(),
            marker: v.get(2).cloned(),
        })
    }).collect())
}

/// Pick minimal relay set: prefer unmarked, fallback to read+write union.
pub fn select_operational_relay_urls(entries: &[RelayListEntry]) -> Vec<String> {
    let unmarked: Vec<_> = entries.iter()
        .filter(|e| e.marker.is_none())
        .map(|e| e.url.clone()).collect();
    if !unmarked.is_empty() { return unmarked; }
    entries.iter()
        .filter(|e| matches!(e.marker.as_deref(), Some("read" | "write")))
        .map(|e| e.url.clone()).collect()
}
```

## FFI-Ready Crate Structure

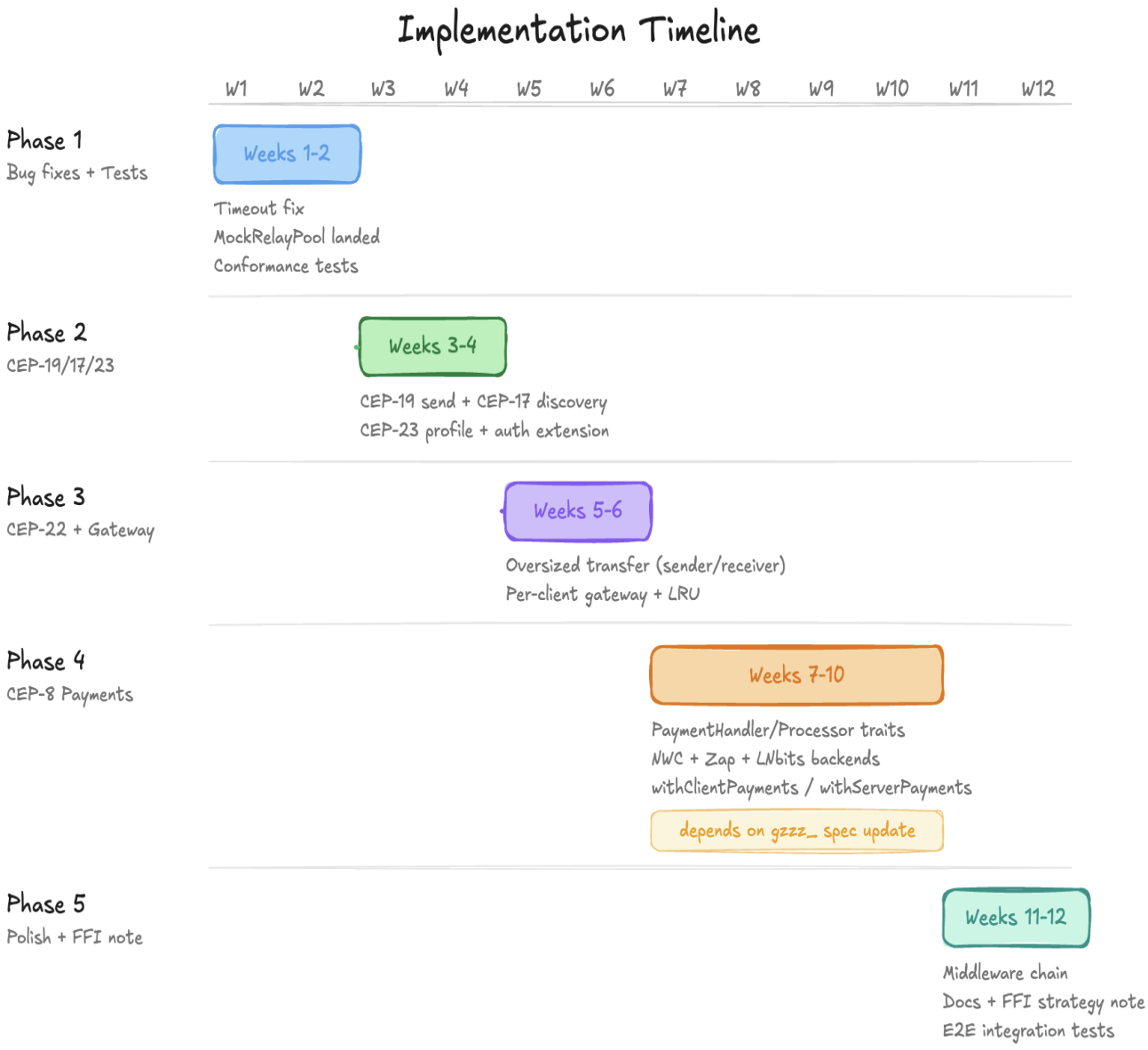


```
contextvm-core  (pure Rust, transport-agnostic)
├── contextvm-nostr  (Nostr transport via nostr-sdk)
├── contextvm-ffi    (C ABI: cbindgen + UniFFI)
│   ├── Kotlin/Swift bindings (mobile)
│   └── C/C++ bindings
├── contextvm-python (PyO3 bindings)
└── contextvm-node   (napi-rs bindings
```

The core crate stays FFI-friendly: no complex generics at public boundaries, simple types throughout, and opaque handles for stateful objects.

## 6. Implementation Steps and Timeline

My timezone is IST (GMT+5:30), and I can dedicate **60 hours per week**, with flexibility to push to **70** if needed. My college has no attendance requirement, so I will be fully available throughout the program. If I complete tasks ahead of schedule, I will contribute to other ContextVM issues and support gzzz.




***Note:*** LNbits *backend is lower priority and will only be tackled if time permits after the core NWC and Zap backends are complete.* ***First priority is to make rust sdk core ready.***

## Phase 1: Bug Fixes + Test Infrastructure (Weeks 1–2)

### Tasks

- **Coordinate on request timeout PR:** An existing PR addresses this. Help land it by investigating how rmcp handles timeouts internally, resolving the mentor's blocker about avoiding redundant timeout logic between the SDK and rmcp.
- **Pick up unauthorized error response fix:** An existing PR includes this fix but is blocked on scope reduction. Pick up the unauthorized error response piece if it gets split out, or implement it independently once the blocked PR's scope is resolved.
- **Land MockRelayPool PR (#30):** Finalize RelayPoolTrait extraction and MockRelayPool. Enables all subsequent integration testing without live relays.
- **Land store extraction PR:** Finalize ClientCorrelationStore and ServerEventRouteStore for cleaner architecture.
- **Conformance test porting:** Complete Phase 1 conformance tests (partially blocked on an in-flight CEP-19 PR, so we work on unblocked tests first).

### Deliverables

- Bug fix PRs merged with regression tests.
- MockRelayPool available for all subsequent phases.
- Expanded conformance test suite.

## Phase 2: CEP-19 + CEP-17 + CEP-23 + Auth Extension (Weeks 3–4)

### Tasks

- **CEP-19 send-side:** Implement kind:21059 creation in the encryption module. An existing PR partially covers this but is blocked on scope reduction. Will coordinate to avoid duplicate work, completing send-side support after the blocked PR lands or implementing independently if it stalls.
- **CEP-17 relay discovery:** Implement `fetchServerRelayList()` and `selectOperationalRelayUrls()`.
- **CEP-23 server profile:** Implement `publishProfileMetadata()` for kind:0 events.
- **isPubkeyAllowed extension:** Add AuthorizationContext, update callback signature, wire through auth flow.
- **Tests:** Unit + conformance tests for each feature, validating wire format with TS SDK.

### Deliverables

- Three CEPs completed and merged.
- Authorization model extended with context-aware callbacks.
- Full test coverage for all new features.

## Phase 3: CEP-22 Oversized Transfer + Per-Client Gateway + Middleware (Weeks 5–6)

### Tasks

- **Middleware chain:** Build the composable message processing pipeline first. CEP-8 payments in Phase 4 are implemented as middleware in the TS SDK (createServerPaymentsMiddleware), so the middleware layer needs to exist before payments can be built on top of it.
- **Oversized transfer module:** Build sender, receiver, and session management in transport/oversized_transfer/.
- **Transport integration:** Hook into client/server transports so that messages >48KB auto-route through chunking.
- **Edge cases:** Abort mid-transfer, timeout on incomplete, chunk ordering, duplicate dedup.
- **Per-client gateway:** Per-pubkey MCP session isolation with LRU eviction.
- **E2E tests:** Oversized flow + multi-client isolation tests over MockRelayPool.

### Deliverables

- CEP-22 complete with sender, receiver, session management.
- Per-client gateway with LRU eviction.
- E2E test coverage for both features.

## Phase 4: CEP-8 Payments (Weeks 7–10)

### Tasks

- **Payment type system:** PaymentRequired/Accepted/Rejected notifications, cap/pmi tag builders.
- **PaymentHandler + NWC handler:** Client-side trait + `LnBolt11NwcPaymentHandler` via nwc crate.
- **PaymentProcessor + NWC processor:** Server-side trait + `LnBolt11NwcPaymentProcessor`.
- **Transport wrappers:** `withClientPayments()` and `withServerPayments()` for automatic payment flow.
- **Zap processor:** `LnBolt11ZapPaymentProcessor` for NIP-57 receipt verification.
- **LNbits backend:** HTTP-based handler/processor via reqwest (stretch goal).
- **Tests:** FakePaymentHandler/Processor stubs + E2E payment flow over MockRelayPool.

#### Deliverables

- Full CEP-8 payment module with NWC and Zap backends.
- Transport wrappers for automatic payment handling.
- LNbits backend (stretch goal).
- Payment flow tests covering key scenarios.

**Note:** CEP-8 depends on gzzz's spec update. If delayed, this phase reorders with Phase 5.

### Phase 5: Polish, Docs, FFI Note (Weeks 11–12)

#### Tasks

- **Integration tests:** Full E2E: gateway→relay→proxy with encryption, payments, oversized transfer.
- **API documentation:** Complete cargo doc with examples. Zero warnings on cargo doc --open.
- **FFI readiness note:** Crate restructuring strategy + UniFFI/PyO3/napi-rs/cbindgen evaluation.
- **Code cleanup:** cargo clippy --deny warnings, cargo fmt, dead code audit.
- **User docs:** README update with examples for all new features.

#### Deliverables

- Polished crate with middleware support.
- Complete API and user documentation.
- FFI strategy document.
- Clean CI (fmt, clippy, doc, test all passing).

## 7. Security and Privacy

### Encryption

- **NIP-44 Encryption:** All encrypted messages use NIP-44 versioned encryption (v2) via nostr-sdk's built-in implementation. The SDK delegates all crypto to nostr-sdk.
- **NIP-59 Gift Wrap:** Encrypted messages are wrapped in gift wrap events (kind 1059/21059) using ephemeral keys, hiding sender identity from relay operators.
- **Encryption Modes:** Three modes: Optional (encrypt if request was encrypted), Required (reject plaintext), Disabled (no encryption). Enforced at transport layer.

### Key Management

- **No key storage:** The SDK never persists private keys. Keys are provided by the application at initialization.

- **Ephemeral keys:** Gift wrapping uses freshly generated ephemeral keys, so the actual sender pubkey is not revealed.
- **Schnorr signatures:** All Nostr events are signed with secp256k1 Schnorr signatures, providing non-repudiation.

### Authorization

- **Pubkey allowlist:** Servers restrict access to configured client pubkeys.
- **Capability exclusions:** Specific methods exempted from allowlist (e.g., tools/list open, tools/call restricted).
- **Dynamic callback:** isPubkeyAllowed enables runtime authorization decisions (database lookups, rate limiting).

### Security Considerations for New Features

- **CEP-8 payment validation:** Server-side processors must validate invoice amounts to prevent underpayment attacks. The PaymentProcessor trait enforces amount checking before marking payment as accepted.
- **No custodial keys:** The SDK never holds Lightning wallet keys. All payment operations go through external wallets via NWC or LNbits, keeping key material out of the SDK process.
- **CEP-22 chunk integrity:** Receivers must validate chunk sequence numbers and total reassembled size against the START frame to prevent injection of extra data or truncation attacks.

### Transport Security

- **Request ID replacement:** Server replaces JSON-RPC IDs with Nostr event IDs internally, preventing cross-client ID guessing.
- **Session isolation:** Per-client gateway ensures MCP state from one client cannot leak to another.
- **Message validation:** All incoming messages validated for size (1MB limit), JSON-RPC 2.0 compliance, and structural correctness.

## 8. Testing Strategy

### Current State

120 test functions across 4 integration test files and 13 inline unit test modules covering core types, serialization, validation, encryption, signer, transport config, and RMCP pipeline.

### Test Infrastructure

- **MockRelayPool (PR #30):** Trait-based relay abstraction with in-memory event storage, subscription matching, and configurable behavior.
- **Conformance tests:** Verify wire format compatibility between Rust and TypeScript SDKs. I'm tracking test porting progress across three issues: #14, #28, and #29.
- **FakePaymentHandler/Processor:** Test stubs for CEP-8 payment flows without Lightning infrastructure.

### Coverage Plan

| Module | Unit Tests | Integration Tests | Conformance |
|---|---|---|---|
| CEP-22 Oversized | Sender, receiver, session | E2E chunking flow | Frame format compat |
| CEP-8 Payments | Types, tag builders, price | E2E payment flow | Notification format |
| CEP-17 Discovery | URL parsing, selection | Relay list fetch | Kind:10002 format |
| CEP-23 Profile | Kind:0 construction | Publish + verify | NIP-01 compliance |
| Auth Extension | Callback with context | Deny/allow flows | Error response format |
| Per-Client Gateway | LRU eviction | Multi-client isolation | - |

### CI Integration

The existing CI workflow (which I contributed) enforces cargo fmt, cargo clippy --deny warnings, and cargo doc on every PR. **All new tests will be integrated into this pipeline,** so that code quality is maintained.

## 9. Documentation Plan

### API Documentation

- Doc comments on all public types, traits, and methods.
- Usage examples in doc comments for key APIs.
- Crate-level documentation with architecture overview.

### Developer Guide

- Architecture overview: TS SDK → Rust module mapping.
- RMCP integration guide: Expose rmcp servers over Nostr.
- Testing guide: Using MockRelayPool for integration tests.
- CEP implementation guide: How each CEP is implemented.

### User Documentation

- README update with examples for all new features.
- New examples in examples/ for payments, oversized, discovery.
- Migration notes for existing users.

### FFI Strategy Document

- Crate restructuring plan (contextvm-core + contextvm-nostr).
- Binding tool evaluation: UniFFI, PyO3, napi-rs, cbindgen.
- API surface recommendations for FFI boundaries.
- Async bridging strategies for non-Rust runtimes.

## 10. Competency Test: Prior Contributions

I have been contributing to the ContextVM rs-sdk (and TS SDK) for the past month. My contributions show familiarity with the codebase, the protocol and the patterns expected by gzzz.

### Merged PRs (rs-sdk, 12 PRs)

| PR | Description | Status |
|---|---|---|
| #6 | Fixed broken cargo build caused by missing util::logger module. Unblocked all development. | **Merged** |
| #7 | Comprehensive tests for core types: EncryptionMode serde, JsonRpcMessage roundtrips, helper methods. | **Merged** |
| #12 | Conformance tests for initialization flow wire format (kind 25910, full handshake). | **Merged** |
| #16 | Wire format tests for tools/list, tools/call, and server announcement (kind 11316). | **Merged** |
| #22 | CI workflow: cargo fmt, clippy --deny warnings, cargo doc checks on every PR. | **Merged** |
| #23 | Signer conformance tests: key derivation, NIP-44 roundtrip, Schnorr signature verification. | **Merged** |
| #26 | Gift-wrap deduplication using LRU cache. Check runs before decryption, IDs inserted after verification. | **Merged** |
| #30 | Extracted RelayPoolTrait and MockRelayPool for integration testing. No sockets, fully deterministic. | **Merged** |

| | | |
|---|---|---|
| #35 | Gift-wrap dedup conformance tests: first/second delivery, ephemeral gift-wrap, LRU eviction. | **Merged** |
| #37 | Extracted ClientCorrelationStore and ServerEventRouteStore. Fixed route preservation and session cleanup race. | **Merged** |
| #38 | Phase 3 integration tests: 8 tests covering e2e init, announcements, encryption, auth, gift-wrap, dedup. | **Merged** |
| #41 | Enriched correlation stores, added SessionStore, ported 45 conformance tests from TS SDK. 219 tests passing. Closes #14 and #28. | **Merged** |

Additionally, I created and maintain three tracking issues (#14, #28, #29) for **conformance test porting** from the TS SDK to the rs-sdk, covering Phase 1, Phase 2, and Phase 3 test coverage.

## 11. Benefits to the Community

- **Feature-Complete Rust SDK:** Closes the 30% feature gap, so anyone building MCP services over Nostr can use Rust with the same capabilities the TypeScript SDK already offers.
- **rmcp Ecosystem Integration:** Existing Rust MCP servers using rmcp's `#[tool]` macros work over Nostr with zero protocol code changes, making adoption much easier.
- **Cross-Language Foundation:** The FFI-ready crate structure enables future bindings for Kotlin/Swift (mobile), Python (ML/AI), and Node.js, expanding ContextVM beyond TypeScript and Rust.
- **Testing Infrastructure:** MockRelayPool and trait abstractions lower the contribution barrier, making it easy for future contributors to write tests without relay infrastructure.
- **Protocol Hardening:** A second independent implementation surfaces spec ambiguities. Conformance tests ensure wire-compatible events between implementations.
- **Always-on Gateways:** Rust gateways can run on low-resource VPS instances without the memory overhead of Node.js, making it cheaper to host persistent MCP services on Nostr.

## 12. About Me

I'm Harsh Chandwani, a 2nd year BS student at Indian Institute of Technology Madras. I started with full-stack web projects about a year and a half ago, then moved toward backend and systems-level work, which led me to Rust. That shift is what brought me to ContextVM and the rs-sdk.

I have **70+ merged open-source contributions** across different projects. On the Bitcoin side, I've worked with rust-bitcoin, Miniscript, and LDK. For ContextVM specifically, I have **10 merged PRs** and 1 open PR on the rs-sdk, plus protocol-level investigation on the TypeScript SDK.

Before writing this proposal I spent time reading the ctxvm draft spec, studying both SDKs, and contributing working code. The specifics in this proposal come from that work and from discussions with gzzz on CVM dev channel.

A full list of my open-source contributions can be found here.

## 13. References

**ContextVM rs-sdk:** https://github.com/ContextVM/rs-sdk

**ContextVM TS SDK:** https://github.com/ContextVM/sdk

**ContextVM Organization:** https://github.com/ContextVM

**ContextVM Specification:** https://contextvm.org

**ContextVM Documentation:** https://docs.contextvm.org

**ContextVM Draft Spec:** https://docs.contextvm.org/spec/ctxvm-draft-spec/

**rmcp (Rust MCP SDK):** https://github.com/modelcontextprotocol/rust-sdk

**nostr-sdk (Rust):** https://github.com/rust-nostr/nostr

**NIP-44 Encryption:** https://github.com/nostr-protocol/nips/blob/master/44.md

**NIP-59 Gift Wrap:** https://github.com/nostr-protocol/nips/blob/master/59.md

**NIP-47 Nostr Wallet Connect:** https://github.com/nostr-protocol/nips/blob/master/47.md

**NIP-57 Zaps:** https://github.com/nostr-protocol/nips/blob/master/57.md

## 14. Availability & Commitment

My college has a 0% attendance requirement, so I can work full time on this regardless of whether the semester is running. During the coding period I can put in 60 hours per week consistently. I am in IST (UTC +5:30) and can adjust my schedule for calls or reviews with gzzz in other timezones.

I have no exams or travel planned during the coding period. If something comes up I will flag it early.

**I am 100% dedicated to ContextVM and have nothing else planned this summer.**

## 15. Post SoB

After Summer of Bitcoin ends I plan to stay involved in a few ways.

CEP-8 payments and CEP-22 oversized transfer will likely need adjustments once they hit real usage. I will keep an eye on issues related to these and fix bugs as they come up.

The rs-sdk's test coverage has improved but still has gaps, especially around integration tests and edge cases. I plan to keep contributing tests and conformance checks after the program ends.

I would also like to continue maintaining the code I wrote during SoB and keep contributing to the ContextVM.